

this is my assignment .give me strategy to solve this in very detail i need to submit it in december . so in very detail explain me what to do

Thought for 11s >

Plan & step-by-step strategy to finish the assignment (due in December)

Nice — this is doable. Below I give a concrete, actionable roadmap that covers **what to implement**, **how to test** each part, **suggested hyperparameters and checks**, what to put in the **report**, and a reproducibility / submission checklist. I follow the assignment text when I refer to required components. sa-server-file-api-user-upload

1 — Read the brief carefully (quick reminder)

- The assignment has two parts: **FrozenLake implementations + experiments** (mandatory) and an **open-ended experiments section**. Your code must produce the `main()` output and an `output.txt` file; the report is max 5 pages; you work in groups of 3. See marks breakdown in the assignment. sa-server-file-api-user-upload
-

2 — Overall deliverables (what you must hand in)

1. `group[group id].zip` containing the folder `group[group id]/`
 - Code files, `output.txt` containing the printed `main()` output.
 2. Report `group[group id].pdf` (≤ 5 pages) answering the questions in §1.7 and describing part 2 experiments.
 3. Individual reflection PDF per student.
-

3 — High-level implementation order (safe ordering for fewer integration headaches)

Work in this order — each step builds on the previous:

1. **Implement FrozenLake environment** (`p`, `r`, `step`, `render`) — necessary for everything else. (If stuck, use `p.npy` as permitted.) sa-server-file-api-user-upload
 2. **Tabular model-based algorithms**: policy evaluation, policy improvement, policy iteration, value iteration.
 3. **Tabular model-free algorithms**: Sarsa, Q-learning (episodic).
 4. **Linear function approximation**: `LinearWrapper`, linear Sarsa, linear Q-learning. Verify tabular $\leftrightarrow$ linear equivalence.
 5. **Deep RL**: `FrozenLakeImageWrapper`, DQN network class, replay buffer, training loop.
 6. **Main function** that runs everything and prints outputs exactly as required.
 7. **Experiments beyond FrozenLake** (choose environment, algorithm(s), metrics).
 8. **Plotting and report preparation** (answer each question in §1.7, include plots & short analysis). sa-server-file-api-user-upload
-

4 — Detailed implementation notes & checks (by component)

4.1 FrozenLake Environment (most critical — 5/50)

- Implement `p(next_state, state, action)` and `r(next_state, state, action)` to match the specification:
 - States indexed row-major; absorbing state is last index (`|s|`).
 - Actions: 0=up, 1=left, 2=down, 3=right.
 - Slip probability `slip`: with probability `slip` the agent moves uniformly in a *random* direction (0.1 in example).
 - Reward: 1 when *taking an action* at the goal (i.e., when state is goal and you act) — and then agent moves to absorbing state. All other rewards 0. Confirm absorbing state transitions always go to itself with 0 reward.
- **Checks**:
 - Load `p.npy` provided and compare your `p` for the small lake — elementwise close. If mismatch, fix indexing/slip handling. (Assignment explicitly suggests `p.npy` as validator). sa-server-file-api-user-upload

- Unit tests: for each tile, check that `sum_next_state_probabilities == 1.0`.
- Run `play(env)` to manually test stepping and rendering.

4.2 Tabular model-based algorithms (policy eval/improve/iteration, value iteration)

- **Policy evaluation:** iterative backup $V(s) \leftarrow \sum_{s'} P(s'|s, \pi(s)) [R(s, s', \pi(s)) + \gamma V(s')]$ until $\max \delta < \theta$.
- **Policy improvement:** greedy with one-step lookahead $\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R + \gamma V(s')]$.
- **Policy iteration:** alternate eval & improvement until policy stable.
- **Value iteration:** synchronous updates $V(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) [R + \gamma V(s')]$.
- **Checks:**
 - Compare your policy/value for small lake to sample output in listing (values/policy shown). Values should be similar (floating point differences OK).
 - Count iterations for big lake (question 1 in §1.7).

4.3 Tabular model-free algorithms: Sarsa & Q-learning

- Implement episodic Sarsa and Q-learning with **ϵ -greedy** where ties are broken *randomly* (important).
- Learning rate & epsilon should **decay linearly** per episode as in the spec (`np.linspace(initial, 0, max_episodes)`).
- For each episode use `env.reset()` and run until `done`.
- **Return storage:** store sum of discounted rewards per episode for plotting (question 2).
- **Checks:**
 - For small lake, run many episodes (e.g., 4k) and verify policy converges to something close to the model-based policy.
 - Plot moving average window 20 of returns (use `np.convolve(..., mode='valid')` per spec).

4.4 Linear function approximation (LinearWrapper + linear Sarsa/Q)

- `LinearWrapper.encode_state(s)` must return a matrix with rows for each action where each row is a one-hot over $|S| * |A|$ with exactly one element=1 at index corresponding to (s,a).

- Theta is a flat vector of length $|S|*|A|$. For the one-hot features, the learning update reduces to the tabular update — show this in report (question 4).
- **Checks:**
 - If you run linear Sarsa / Q with one-hot features and same hyperparameters as tabular, results should match tabular algorithms (up to random seeds).

4.5 Deep Q-Network

- `FrozenLakeImageWrapper` :
 - Return shape $(4, h, w)$: channel 0 = agent position; channel 1 = start map; channel 2 = holes map; channel 3 = goal map. For absorbing state, return all zeros or stacked zeros per spec (there is a mapping in the listing).
- `DeepQNetwork.forward` :
 - Convert input list/np.array to `torch.tensor` with shape $(B, 4, h, w)$, pass through Conv2d -> ReLU -> flatten -> Linear -> ReLU -> output Linear -> Q-values shape $(B, |A|)$.
- `train_step` :
 - Build tensors for `q` (gather chosen actions), compute `next_q = tdqn(next_states).max(dim=1)[0] * (1-dones)` , `target = rewards + gamma * next_q` .
 - Loss = MSE between `q` and `target` . Backprop and `optimizer.step()` .
- `ReplayBuffer.draw` returns random sample without replacement (use `random_state.choice` indices).
- DQN loop: epsilon-greedy using network (break ties randomly), append transitions, when `len(buffer) >= batch_size` draw batch and train; update target every `target_update_frequency` .
- **Checks:**
 - For tiny conv layers / network sizes in `main()` example, DQN may not reach same performance — that's expected. Make sure training runs without NaNs.
 - Plot training return curves similar to tabular algorithms.

5 — Testing, debugging & tips

- Seed everything (numpy, torch, random) to make results reproducible for the report.
- Sanity tests:

- Deterministic zero-slip variant: if `slip=0`, deterministic transitions — ensure policy/value match expected shortest path.
 - For absorbing state `s_absorb`, $p(s_absorb|s_absorb,a)=1$ and reward 0.
 - **Common bugs:**
 - Off-by-one state indexing (row-major mapping).
 - Incorrect slip distribution (include desired direction in random slip choices).
 - Not making `done` true when absorbing state reached.
 - In ϵ -greedy: not randomizing among ties.
 - **Profiling:** if training is slow, reduce episodes while debugging. For final runs, use full episodes.
-

6 — Experiments to include (Part 1 questions and marks)

Follow §1.7 when making experiments and plots. Key items to include:

1. **Iterations to convergence** for big lake: report number for policy iteration and value iteration (question 1). sa-server-file-api-user-upload
 2. **Return curves:** for each model-free method (Sarsa, Q, linear variants, DQN) store per-episode discounted return. Plot moving average (window 20) vs episode number. Label axes; put legend. (question 2). sa-server-file-api-user-upload
 3. **Hyperparameter tuning experiments:** attempt to minimize episodes to optimal policy on the small lake by changing `eta` (learning rate initial) and `epsilon` initial. Record trials and show best settings; then try to find policy for big lake and describe results (question 3). sa-server-file-api-user-upload
 4. **Theory answers:** short textual answers to Q4–Q6 (linear interpretation, ϵ -greedy necessity, target network reason per Mnih et al.). Keep these concise and referenced.
-

7 — Suggested hyperparameters / practical choices (starting points)

- `gamma = 0.9` (assignment). sa-server-file-api-user-upload
- Tabular Sarsa/Q: `max_episodes = 4000` (assignment example), initial `eta=0.5`, `epsilon=0.5`, linearly decay to 0. Use seeds. sa-server-file-api-user-upload

- DQN defaults from main: `learning_rate=0.001` , `epsilon=0.2` , `batch_size=32` , `buffer_size=256` , `target_update_frequency=4` , `conv kernel size 3` , `conv_out_channels=4` , `fc_out_features=8` , `seed=4` . These are in the assignment sample; use them unless you tune. sa-server-file-api-user-upload

8 — Code organization (recommended)

bash

 Copy code

```
group[group id]/
├─ frozenlake.py           # EnvironmentModel, FrozenLake, play()
├─ model_based.py         # policy_eval, policy_improvement, policy_iteration, va
├─ tabular_rl.py          # sarsa, q_learning
├─ linear_rl.py           # LinearWrapper, Linear_sarsa, Linear_q_Learning
├─ dqn.py                 # FrozenLakeImageWrapper, DeepQNetwork, ReplayBuffer, d
├─ main.py                # main() that runs everything and writes output.txt
├─ utils.py               # plotting, seeds, small helpers
├─ p.npy                  # (if you use it for validation)
└─ requirements.txt
```

9 — Report structure (≤ 5 pages) — exactly what to put

1. Title + group + student numbers (1 line).
 2. Short implementation summary: files, language, main design decisions (0.5 page).
 3. Part 1 — Results:
 - Short methodology (how you implemented each algorithm).
 - Key results: policy iterations count (Q1), plots of moving average returns for each model-free method (Q2). Show best hyperparameter trials (Q3). Include a concise answer to Q4–Q6. (≈ 2 pages) sa-server-file-api-user-upload
 4. Part 2 — Chosen env + algo: describe environment, algorithm and experimental protocol, results and interpretation (≈ 1 page).
 5. Reproducibility & final remarks: how to run code and reproduce plots, seeds used.
-

10 — Reproducibility checklist before submission

- `output.txt` created by running `python main.py` — contains printed outputs for all required algorithms. (This is checked for marks.) sa-server-file-api-user-upload
 - `requirements.txt` with exact package versions (numpy, torch version used).
 - All random seeds fixed in main.
 - Report answers Q1–Q6 explicitly.
 - Individual reflections submitted.
-

11 — Short pointers on what to write for the theory questions

- **Q1:** count iterations from your runs; include both values and brief comment on why one might converge faster. (policy iteration does full policy evals; value iteration directly updates values—observed counts). sa-server-file-api-user-upload
 - **Q2:** show 5 plots (Sarsa, Q, linear Sarsa, linear Q, DQN) — moving average window 20. Explain smoothing and interpretation. sa-server-file-api-user-upload
 - **Q3:** report grid search over $\eta \in \{0.1, 0.3, 0.5\}$ and $\epsilon \in \{0.1, 0.3, 0.5\}$ or similar; show best pair and episodes-to-optimal (or failure). For big lake report whether you could reach optimal and what changes helped/hindered.
sa-server-file-api-user-upload
 - **Q4:** with one-hot features, θ component corresponds to $Q(s,a)$ directly; tabular algorithms = linear approximator with one-hot features. (Short formal explanation + 2 sentences). sa-server-file-api-user-upload
 - **Q5:** ϵ -greedy ensures ongoing exploration to avoid premature convergence, especially when Q estimates are noisy / network is updated from off-policy samples. (Short).
sa-server-file-api-user-upload
 - **Q6:** target network stabilizes learning by giving a slowly changing target for TD updates (reduces oscillations/divergence) — cite Mnih et al. briefly. sa-server-file-api-user-upload
-

12 — Quick debugging recipes (if results look bad)

- If model-based results wrong → check `p / r` against `p.npy`. sa-server-file-api-user-upload

- If tabular RL fails to learn → check ϵ -greedy implementation & random tie breaking; check discounting when computing episode return.
- If linear approx differs from tabular → verify feature encoding is exactly one-hot in same indexing.
- If DQN diverges → reduce learning rate, check done masking for next Q (use (1-done)), ensure target updates are periodic, check replay sampling and normalization of inputs.

13 — Minimal runnable checklist to get marks quickly

If time is short, prioritize:

1. Correct FrozenLake env (5 points). Validate with `p.npy` . sa-server-file-api-user-upload
 2. Policy iteration & value iteration (7.5 points).
 3. Sarsa & Q-learning tabular (7.5 points) with stored returns & plots (question 2).
 4. Write `main()` to call the above and create `output.txt` .
 5. Finish report answering Q1–Q3 concisely.
- You will then have a solid submission; build linear & DQN if you have extra time.

14 — Example commands & run script snippet

Put this in `main.py` (simplified):

py

 Copy code

```
if __name__ == "__main__":
    env = FrozenLake(lake, slip=0.1, max_steps=16, seed=0)
    gamma=0.9
    policy_pi, value_pi = policy_iteration(env, gamma, theta=1e-3, max_iterations=
    env.render(policy_pi, value_pi)
    # run sarsa, q-learning and save returns arrays to files for plotting
    # run linear wrappers, dqn
    # print outputs and also write same prints to output.txt
```


15 — Final advice & next immediate steps (what to do right now)

1. Clone the recommended code layout and implement `FrozenLake` first. Validate against `p.npy` . sa-server-file-api-user-upload
2. Implement model-based algorithms and confirm their output for the small lake matches the listing samples.
3. Add model-free tabular algorithms and start collecting episode returns. Plot moving averages.
4. Iterate to linear and DQN if time permits.
5. Prepare the report while running final long experiments.

If you want, I can now:

- generate a **starter code skeleton** (files with function stubs in the recommended layout) so you can copy/paste and start implementing; or
- produce the **exact unit tests** to validate your `p` function against `p.npy` , or
- produce a short **example main.py** that runs policy iteration, value iteration and writes `output.txt` (print formatting matching assignment).

Tell me which of those you'd like me to produce right away and I'll create it.



explain me the core idea of game in easy language

Here is the **core idea of the FrozenLake game**, explained in **very easy language**, so you fully understand what the game actually is before doing all the maths and algorithms:

What is FrozenLake? (Super Easy Explanation)

Think of FrozenLake as a **grid game** — like walking on a frozen lake made of tiles.

Each tile can be:

- **S** → Start
- **F** → Frozen tile (safe)
- **H** → Hole (you fall and the game ends → BAD)
- **G** → Goal (you reach the treasure → GOOD)

You (the agent) start at **S**.

Your job:

➡ **Move step-by-step to reach G without falling into any H.**

You can move:

- **UP** (0)
- **LEFT** (1)
- **DOWN** (2)
- **RIGHT** (3)

BUT the lake is slippery! (Main twist of the game)

This is the **most important part**:

Even if you choose a direction,
you might **slip** and move in a different direction.

Example:

You choose **RIGHT** →

but with 10% chance, you actually go a **random direction** instead.

This makes the game **uncertain and risky**, so the agent must learn a strategy that works **on average**, not just in perfect conditions.

How you win or lose

- Reach **G** → you get reward = 1
- Fall into **H** → **game ends** (reward 0)
- Walk on frozen tiles → **reward 0**

- If you reach the goal, you are sent to an **absorbing state** → like "Game Over room" where nothing more happens.
-

What are we trying to learn?

We want the **agent** (your algorithm) to learn:

★ A policy

= The **best action** to take in every tile so that the agent **maximizes the chance of reaching G**.

Because the lake is slippery, the best policy is not just the shortest path — it's the **safest reliable path**.

Why this game is useful for Reinforcement Learning

FrozenLake teaches your algorithms:

- How to deal with **uncertainty**
- How to **plan ahead**
- How to **evaluate states** (good/bad positions)
- How to **learn from trial-and-error**
- How to balance **exploration** (trying new moves) and **exploitation** (using what you know)

This is why the assignment makes you implement:

- Policy iteration
- Value iteration
- Sarsa
- Q-learning
- DQN

Because all of these need a simple but tricky environment to learn from.

Core Idea in 1 Sentence

FrozenLake is a game where you try to reach the goal without falling into holes, but the ground is slippery so you can't fully control your movements — and your algorithm must learn a strategy that works despite this uncertainty.

If you want, I can also explain:

- ✓ Why slip makes RL harder
- ✓ The difference between small lake and big lake
- ✓ How rewards work mathematically
- ✓ Why an absorbing state is used

Just tell me!



yes explain me like a teacher

Absolutely — I'll explain FrozenLake **like a teacher explaining to a beginner**, step-by-step, with simple examples and zero technical jargon at first.

By the end, you'll have a *strong intuitive understanding* of the environment, which makes the algorithms much easier to implement later.

🌟 FrozenLake — Teacher-Style Explanation

Imagine you and I are in a classroom.

I draw a small grid on the board:

r

 Copy code

```
S F F F
F H F H
F F F H
H F F G
```

Then I tell you:

"This is a frozen lake. You start at **S**. You want to reach **G**.
But the ice is slippery. If you step wrong, you can fall into a **hole (H)**."

Let's go through the ideas one by one.



1. What is the world made of?

The lake is made of **cells** (small squares).

Each cell is one of four types:

1. **S — Start**

Where the agent begins.

2. **F — Frozen tile**

Normal safe ground.

3. **H — Hole**

If you step here → you fall → game ends.

4. **G — Goal**

The treasure / the final destination.

That's it. The entire game is this little world.



2. What can the agent do?

The agent (your AI) can choose one of four actions:

- **UP**
- **LEFT**
- **DOWN**
- **RIGHT**

So far, easy.

3. But the ice is slippery! (This is the twist)

Imagine walking on a frozen lake in real life —
even if you want to walk forward, your foot may slip.

That's exactly what happens here.

Example:

You choose **RIGHT**.

But due to slipperiness (say 10% slip chance):

- 90% chance → you go **RIGHT** (as intended)
- 10% chance → you slip and go in a **random** direction

So the game is **uncertain**.

You don't always get what you want.

This is what makes the game interesting for RL.

4. How rewards work

In this game:

- Reaching **G** (the goal) = **reward 1**
- Everything else = **reward 0**

You do **not** get punished for falling in a hole.

You just get **0** and the game ends.

This means:

Your algorithm must *figure out by itself*
that falling in holes is bad
because it prevents it from ever reaching the goal.

5. How the game ends

There are only two ways the game can end:

1. You fall into a hole → END

2. You reach the goal → END

After the game ends, the agent is placed in an **absorbing state**.

I explain it like this:

"Imagine a special room where nothing happens.

Once you enter it, you stay there forever until a new game starts."

This absorbing state is important because it helps algorithms handle end-of-episode situations properly.

6. What is the agent trying to learn?

The whole goal of Reinforcement Learning here is:

👉 To find the best path through the frozen lake

"Best" means:

- Safest
- Most reliable
- Highest chance of reaching the goal
- Even with slipperiness (uncertainty!)

The agent gradually learns a **policy**, which means:

"If I am at this tile, what action should I take?"

A policy is like a map of instructions:

CSS

 Copy code

From here go RIGHT.

From here go UP.

From here go DOWN.

7. Why do we use FrozenLake in RL assignments?

Because FrozenLake is:

- Small
- Simple to understand
- But tricky enough to challenge algorithms
- Contains randomness
- Has delayed rewards
- Has dead ends (holes)
- Has an optimal strategy that is **not obvious**

It lets you test:

- Policy iteration
- Value iteration
- Sarsa
- Q-Learning
- Linear approximation
- DQN (Deep Q Network)

All on the **same environment**.



8. How the agent learns

Think of the agent like a student learning by trial and error.

At first:

- It wanders randomly
- Falls into many holes
- Barely reaches the goal

But over thousands of trials:

- It remembers what worked
- It avoids dangerous zones
- It becomes confident
- It finds the safest path

Eventually, it develops a strategy like:

"Going straight to the goal is risky.
Better to take a longer but safer path."

This is exactly what RL is about.

FrozenLake in One Sentence

It's a grid world where you try to reach a goal without falling into holes, but because the ice is slippery, you can't fully control your movement — so your AI must learn a reliable plan despite uncertainty.

If you want, I can now explain:

- ✓ How to mathematically represent the game
- ✓ How states are numbered
- ✓ What transition probabilities look like
- ✓ How to write the $p()$ and $r()$ functions
- ✓ A drawing of the grid and how agent moves
- ✓ Why absorbing state is needed
- ✓ What an optimal policy looks like

Just tell me — I'll explain each part like a teacher too.

    ...

yes explain me

Perfect — I will continue **like a teacher**, step-by-step, and explain the next core parts:

1. How states are numbered

2. How transitions work (with examples)
3. How to build the $p(s'|s,a)$ table
4. How rewards $r(s,a,s')$ work
5. Why we need an absorbing state

These are the most important parts for implementing the assignment correctly.



1. How states are numbered (very important for coding)

The FrozenLake grid is like this:

r

 Copy code

```
S F F F
F H F H
F F F H
H F F G
```

This is a 4×4 grid = 16 cells.

We number the states using **row-major order** (left-to-right, top-to-bottom).

Teacher explanation:

“Imagine reading rows like reading a book.”

So the numbering becomes:

 Copy code

```
0  1  2  3
4  5  6  7
8  9 10 11
12 13 14 15
```

If the lake were bigger, you follow the same rule.



Formula to convert grid position → state number

If a cell is at:

- $\text{row} = r$
- $\text{column} = c$
- $\text{number of columns} = n$

Then

👉 **$\text{state} = r * n + c$**

This formula is very important when writing your code.

2. How the agent moves (actions)

The four actions are:

- 0 = UP
- 1 = LEFT
- 2 = DOWN
- 3 = RIGHT

Teacher explanation:

“Actions do NOT directly move the agent.

The ice might make you slip, so the final movement may be different.”

3. Slip probability — how randomness works

Assume $\text{slip} = 0.1$ (10%).

This means:

- 90% → you move in the direction you chose
- 10% → you slip and move in a **random direction**

So if $\text{action} = \text{RIGHT}$:

- 90% → go RIGHT
- 10% → go UP / LEFT / DOWN / RIGHT (equally likely)

If slipping gives 4 random directions with equal probability:

- Each random direction gets 1/4 of the 10%
- So each slip direction has probability = 0.025

4. How transitions work (Teacher explanation with example)

Let's say you are at **state 2** (row 0, column 2):

```
r
```

 Copy code

```
S F YOU F
```

You choose action = RIGHT.

◆ Case 1: You move as intended (90%)

From state 2:

RIGHT means → state 3.

So:

```
sql
```

 Copy code

```
p(3 | 2, RIGHT) += 0.9
```

◆ Case 2: You slip (10%)

Slip chooses from [UP, LEFT, DOWN, RIGHT] each with probability = 0.025.

We calculate each direction:

Slip UP from state 2:

UP → row -1 = invalid → stay in same state (2)

Slip LEFT:

LEFT → state 1

Slip DOWN:

DOWN → state 6

Slip RIGHT:

RIGHT → state 3

So add 0.025 each to these states.



5. Building the transition matrix $p(s'|s,a)$

Your p function should return the probability of ending in each next state.

Since next state depends only on (s, a) , p is a **3D table**:

CSS

 Copy code

```
p[next_state][state][action]
```

For a 16-state grid and 4 actions, p is $16 \times 16 \times 4$.

Teacher explanation:

“For every state and action, we must fill in the probabilities of landing in every possible next state.”

Even if a transition has 0 probability, it must still be included.



6. Rewards $r(s, a, s')$

Reward rules (super easy):

- If you reach the **goal** (G) → reward 1
- All other transitions → reward 0

Important:

You get the reward **when you take an action from the goal tile**, not when you enter it.

After reaching the goal, you go to the **absorbing state**.

7. What is an absorbing state and why do we need it?

Teacher explanation:

"Imagine a special room where nothing else can happen.
Once you finish the game, we put you there so the agent
doesn't continue wandering."

In the absorbing state:

- All actions keep you in the same state
- Reward is always 0
- Game is effectively over

Why do we need it?

- RL algorithms expect a fixed number of steps
- It simplifies the maths
- Makes value iteration and policy iteration easier
- DQN also expects stable episode endings

So after reaching G or falling into H → agent moves to absorbing state.

8. Putting it all together (Example)

Let's simulate one step:

You are at state 6 (middle of the grid):

powershell

 Copy code

```
. . .  
. H .  
. . .
```

You choose DOWN.

If you move normally → you go to state 10.

If you slip → any of 4 directions possible.

You compute probabilities exactly as explained.

If next position is a HOLE:

- next state = absorbing state
- reward = 0

If next position is GOAL:

- next state = absorbing state
- reward = 1

Otherwise:

- next state = new tile
 - reward = 0
-

9. Final summary (teacher style)

Here's the whole logic in short:

1. The lake is a grid of safe tiles, holes, and a goal.
 2. States are numbered row by row.
 3. The agent can take 4 actions.
 4. Because the lake is slippery, actions are not always followed exactly.
 5. The transition probability matrix $p(s'|s,a)$ describes how likely you are to end up in each next state.
 6. You get reward 1 only when leaving the goal tile.
 7. After goal or hole, you go to an absorbing state.
 8. The goal of RL is to learn the safest, most reliable path to reach G with high probability.
-

If you want, I can now explain:

- ✓ EXACTLY how to implement $p()$ and $r()$ in Python
- ✓ How to encode actions correctly
- ✓ How to convert between coordinates and state numbers
- ✓ How to validate your environment using `p.npy`

✓ Example code template for the FrozenLake class

Just tell me “Next step please” and I’ll continue like a teacher.



next step pleaseee

Great!

Now we move to the **next step**, again explained like a teacher:

NEXT STEP: How to implement `p()` and `r()` in your FrozenLake environment

This is the **heart of the whole assignment** — if this is correct, all algorithms (policy iteration, Q-learning, DQN, linear) will work correctly.

We will go slowly, step-by-step.

1. First, understand what `p()` MUST return

Your assignment says `p()` must return:

CSS

 Copy code

```
p(next_state, state, action)
```

Meaning:

If I am at `state` and I take `action`, what is the probability of going to `next_state`?

`p` is like a **big 3D table**.

If your lake has `N` states and 4 actions, then:

mathematica

 Copy code

```
p is shape (N, N, 4)
```

- First dimension: `next_state`

- Second dimension: current_state
- Third dimension: action

But you do NOT need to build the full 3D table at once.

You only need a function:

python

 Copy code

```
def p(self, next_state, state, action):  
    return probability
```

This function will compute the probability on the fly.

2. How to convert “state number” $\leftrightarrow$ (row, col)

You must be able to convert between **state index** and **grid coordinates**.

State $\rightarrow$ (row, col)

python

 Copy code

```
row = state // ncols  
col = state % ncols
```

Example for 4×4 lake:

State 6 $\rightarrow$ (1, 2)

State 14 $\rightarrow$ (3, 2)

(row, col) $\rightarrow$ state number

python

 Copy code

```
state = row * ncols + col
```

This is essential for building transitions.

3. How to compute the intended movement (if no slip)

We have 4 actions:

- 0 = UP $\rightarrow$ (row - 1, col)
- 1 = LEFT $\rightarrow$ (row, col - 1)
- 2 = DOWN $\rightarrow$ (row + 1, col)
- 3 = RIGHT $\rightarrow$ (row, col + 1)

When an action would go outside the grid, you **stay in the same state**.

Example:

If you are at state 1 (0,1) and you choose UP $\rightarrow$ new row = -1 $\rightarrow$ invalid $\rightarrow$ stay at 1.

4. Slip logic (SUPER IMPORTANT)

If slip = 0.1:

- 90% $\rightarrow$ move in intended direction
- 10% $\rightarrow$ slip into a random direction (4 options equally likely)

Slip probability per direction:

```
ini
```

 Copy code

```
slip_prob_each = slip / 4
```

So:

For each action:

- Intended direction: prob = 1 - slip
- UP: prob += slip_prob_each
- LEFT: prob += slip_prob_each
- DOWN: prob += slip_prob_each
- RIGHT: prob += slip_prob_each

But each UP/LEFT/DOWN/RIGHT becomes a separate **next state**, depending on valid moves.

5. Handling holes & goals $\rightarrow$ ALL go to absorbing state

If your **current tile is hole OR goal**, any action leads to absorbing state, with probability 1.

If your **next tile is a hole**, then:

- `next_state` = absorbing state
- `reward` = 0

If your **next tile is the goal**, then:

- `next_state` = absorbing state
- `reward` = 1

Absorbing state index = last state:

```
ini
```

 Copy code

```
absorbing = n_states - 1
```

6. Reward function $r(s,a,s')$

Simple rules:

● **If (state is goal) and you take an action $\rightarrow$ reward = 1**

(Then go to absorbing)

● **Else $\rightarrow$ reward = 0**

(Anything else including hole $\rightarrow$ reward 0)

7. TEACHER-STYLE: Step-by-step pseudocode for `p()`

Here is the **logic**, explained like a teacher would write on the whiteboard:

```
python
```

 Copy code

```
FUNCTION p(next_state, state, action):
```

```
    IF state is absorbing:
```

```
        IF next_state == absorbing:
```

```
            return 1
```

```

ELSE:
    return 0

IF state is HOLE or GOAL:
    IF next_state == absorbing:
        return 1
    ELSE:
        return 0

# 1) find row, col of current state
(row, col) = convert state to coordinates

total_prob = 0

# 2) For each possible move direction (UP, LEFT, DOWN, RIGHT):
FOR dir in [0,1,2,3]:

    # find prob of taking this direction
    IF dir == action:
        prob = (1 - slip) + slip/4
    ELSE:
        prob = slip/4

    # find where this direction goes
    (new_row, new_col) = row + delta_row[dir], col + delta_col[dir]

    IF (new coordinates invalid):
        new_state = state # stay in same place
    ELSE:
        new_state = convert to state number

    # holes go to absorbing
    IF new_state is HOLE:
        new_state = absorbing

    # goals go to absorbing
    IF new_state is GOAL:
        new_state = absorbing

    IF new_state == next_state:

```

```
total_prob += prob

return total_prob
```

This logic automatically adds all correct probabilities.

✨ 8. FULL PYTHON CODE TEMPLATE (copy/paste ready)

Here is the exact Python function you will use inside your class:

python

 Copy code

```
def p(self, next_state, state, action):

    absorbing = self.n_states - 1 # index of absorbing state
    slip = self.slip

    # If state is absorbing
    if state == absorbing:
        return 1.0 if next_state == absorbing else 0.0

    # If state is hole or goal
    if self.lake[state] in ('H', 'G'):
        return 1.0 if next_state == absorbing else 0.0

    # Convert state → (row, col)
    row = state // self.ncols
    col = state % self.ncols

    # Movements: up, left, down, right
    moves = [(-1,0), (0,-1), (1,0), (0,1)]

    prob = 0.0
    slip_each = slip / 4

    for a in range(4): # for each direction
        if a == action:
            p_dir = (1 - slip) + slip_each
        else:
            p_dir = slip_each
```

```

dr, dc = moves[a]
nr, nc = row + dr, col + dc

# check boundary
if 0 <= nr < self.nrows and 0 <= nc < self.ncols:
    ns = nr * self.ncols + nc
else:
    ns = state # out of bounds → stay

# hole → absorbing
if self.lake[ns] == 'H':
    ns = absorbing

# goal → absorbing
if self.lake[ns] == 'G':
    ns = absorbing

if ns == next_state:
    prob += p_dir

return prob

```

You can paste this directly into your `FrozenLake` class and adapt variable names.

9. Reward function template too

python

 Copy code

```

def r(self, next_state, state, action):

    absorbing = self.n_states - 1

    # Taking action from goal gives reward 1
    if self.lake[state] == 'G':
        return 1.0

```

```
# Everything else gives reward 0  
return 0.0
```

10. What we have learned so far

You now understand:

- ✓ How states are numbered
- ✓ How actions map to movements
- ✓ How slip affects transitions
- ✓ How to compute probabilities
- ✓ How hole/goal transitions work
- ✓ How to implement `p()` step-by-step
- ✓ How to implement reward function `r()`
- ✓ Why absorbing state is needed

These are the **foundation** of the whole project.

NEXT STEP (choose one):

Tell me which you want next:

- 1 How to implement `step()` function
- 2 How to implement `FrozenLake.render()`
- 3 How to implement policy evaluation
- 4 How to implement policy iteration
- 5 How to implement Q-learning and Sarsa
- 6 How to implement LinearWrapper
- 7 How to implement DQN
- 8 How to validate your environment using `p.npy`

9 How to answer Q1–Q6 in the report

Just type the number(s).



give every thing step by step like a teacher

Thought for a couple of seconds >

Step-by-step teacher-style guide — start-to-finish (everything you need)

I'll walk you through **every piece** of the assignment, step-by-step — from the environment implementation up to final experiments, testing, and the report. Read each section and implement it in that order. I give you *why* each step matters, *how* to do it, and ready-to-copy code templates you can adapt.

1. Project layout (create these files first)

Create a single folder for submission (e.g., `group42/`) with this structure:

bash

Copy code

```
group42/
├─ frozenlake.py          # env (p, r, step, render)
├─ model_based.py         # policy_eval, policy_improve, value_iteration, policy_i
├─ tabular_rl.py          # sarsa, q_learning
├─ linear_rl.py           # LinearWrapper, Linear_sarsa, Linear_q_Learning
├─ dqn.py                 # FrozenLakeImageWrapper, DeepQNetwork, ReplayBuffer, dq
├─ utils.py               # helpers: seed, plotting, save/load
├─ main.py                # orchestrates runs and writes output.txt
├─ p.npy                  # (provided if you use it to validate)
├─ requirements.txt
└─ README.md
```


Keep code modular: each file should implement functions/classes described in its name.

2. Implement the FrozenLake environment (frozenlake.py)

2.1 What the class must do

- Hold the lake map and dimensions.
- Provide `p(next_state, state, action)` returning probability.
- Provide `r(next_state, state, action)` returning reward.
- Provide `reset()` → returns state index (start).
- Provide `step(action)` → returns `(next_state, reward, done, info)`.
- Provide `render()` for debugging (text output).
- Keep an absorbing state as the final index.

2.2 Core data members

- `self.lake` — list[str] of tiles (length N-1), then absorbing state implicitly as last index.
- `self.nrows, self.ncols`
- `self.n_states = nrows*ncols + 1` (if you add absorbing as extra) *or* use `nrows*ncols` where last index is absorbing (be consistent).
- `self.start_state` index.
- `self.slip` float in [0,1].
- `self.max_steps` optional.
- `self.state` current state.
- `self.absorbing` index (typically last: `n_states-1`).

2.3 Copy-paste-ready implementation templates

Note: adjust small names to match your codebase. This code is a clear, teacher-level template.

python

 Copy code

```
# frozenLake.py
import numpy as np
import random
```

```

class FrozenLake:
    def __init__(self, lake_grid, nrows, ncols, slip=0.1, start=None, max_steps=10, seed=None):
        """
        lake_grid: list of characters length nrows*ncols (S,F,H,G)
        nrows, ncols: int
        slip: float probability of slipping
        start: index of start state (if None, find 'S')
        """

        self.nrows = nrows
        self.ncols = ncols
        self.lake = lake_grid[:] # len = nrows*ncols
        self.slip = float(slip)
        self.max_steps = max_steps
        self.seed = seed
        if seed is not None:
            random.seed(seed)
            np.random.seed(seed)

        # We'll use last index as absorbing state index = nrows*ncols
        self.n_tiles = nrows * ncols
        self.absorbing = self.n_tiles # index for absorbing state
        # Optionally append an entry (not used for map lookups)
        self.n_states = self.n_tiles + 1

        # find start
        if start is None:
            self.start = next(i for i, c in enumerate(self.lake) if c == 'S')
        else:
            self.start = start

        self.reset()

    def reset(self):
        self.state = self.start
        self.steps = 0
        return self.state

    def to_rc(self, state):
        return state // self.ncols, state % self.ncols

```

```

def to_state(self, r, c):
    return r * self.ncols + c

def is_terminal_tile(self, tile_index):
    return self.lake[tile_index] in ('H', 'G')

def p(self, next_state, state, action):
    # Implementation as taught
    if state == self.absorbing:
        return 1.0 if next_state == self.absorbing else 0.0

    # If current tile is H or G -> go to absorbing
    if self.lake[state] in ('H', 'G'):
        return 1.0 if next_state == self.absorbing else 0.0

    row = state // self.ncols
    col = state % self.ncols
    moves = [(-1,0), (0,-1), (1,0), (0,1)]
    slip_each = self.slip / 4.0

    prob = 0.0
    for a in range(4):
        p_dir = (1.0 - self.slip + slip_each) if a == action else slip_each
        dr, dc = moves[a]
        nr, nc = row + dr, col + dc
        if not (0 <= nr < self.nrows and 0 <= nc < self.ncols):
            ns = state
        else:
            ns = self.to_state(nr, nc)
            if self.lake[ns] == 'H' or self.lake[ns] == 'G':
                ns = self.absorbing

        if ns == next_state:
            prob += p_dir
    return prob

def r(self, next_state, state, action):
    # Reward 1 if action from goal; else 0
    if state < self.n_tiles and self.lake[state] == 'G':
        return 1.0

```

```

    return 0.0

def step(self, action):
    # returns next_state, reward, done, info
    if self.state == self.absorbing:
        return self.absorbing, 0.0, True, {}

    # compute distribution over next states
    # We'll sample: choose dir based on slip
    rand = random.random()
    if rand < (1 - self.slip):
        chosen_dir = action
    else:
        chosen_dir = random.randint(0, 3)

    row, col = self.to_rc(self.state)
    moves = [(-1,0), (0,-1), (1,0), (0,1)]
    dr, dc = moves[chosen_dir]
    nr, nc = row + dr, col + dc
    if not (0 <= nr < self.nrows and 0 <= nc < self.ncols):
        ns = self.state
    else:
        ns = self.to_state(nr, nc)
        if self.lake[ns] in ('H', 'G'):
            ns = self.absorbing

    reward = 0.0
    # reward logic: reward=1 if current tile was goal (as defined)
    if self.state < self.n_tiles and self.lake[self.state] == 'G':
        reward = 1.0

    self.state = ns
    self.steps += 1
    done = (self.state == self.absorbing) or (self.steps >= self.max_steps)
    return self.state, reward, done, {}

def render(self):
    # simple text render showing S,F,H,G and an A for agent (if not in absorb
    grid = []
    for i, c in enumerate(self.lake):

```

```

        if i == self.state:
            grid.append('A') # agent
        else:
            grid.append(c)
# print rows
for r in range(self.nrows):
    row = grid[r*self.ncols:(r+1)*self.ncols]
    print(' '.join(row))
if self.state == self.absorbing:
    print("Agent in absorbing state.")

```

Teacher tip: Keep `self.absorbing` consistent across `p`, `r`, and `step`. Use the same indexing everywhere.

3. Unit tests & validation (validate `p` with `p.npy`)

Create a tiny test script to compare your `p` function to the provided `p.npy` (assignment suggested validator). This will catch indexing/slip mistakes.

python

 Copy code

```

# test_p.py
import numpy as np
from frozenlake import FrozenLake

p_reference = np.load('p.npy') # shape probably (N, N, 4) depending on provided
lake = ... # Load your lake from assignment file
env = FrozenLake(lake, nrows=..., ncols=..., slip=0.1)
N = env.n_states
max_diff = 0.0
for ns in range(N):
    for s in range(N):
        for a in range(4):
            myp = env.p(ns, s, a)
            ref = p_reference[ns, s, a]
            diff = abs(myp - ref)
            if diff > max_diff:
                max_diff = diff

```

```
print("Max diff:", max_diff)
assert max_diff < 1e-6, "p() doesn't match reference"
```

If this fails: likely off-by-one indexing or absorbing index definition mismatch. Inspect small entries to debug.

4. Implement policy evaluation, improvement, iteration, and value iteration (model_based.py)

4.1 Policy evaluation (iterative)

- Input: env, policy (array length n_states with actions), gamma, theta
- Iteratively update $V(s)$ until $\max_delta < \theta$.

python

 Copy code

```
def policy_evaluation(env, policy, gamma=0.9, theta=1e-6, max_iter=1000):
    V = np.zeros(env.n_states)
    for i in range(max_iter):
        delta = 0.0
        for s in range(env.n_states):
            v = V[s]
            a = policy[s]
            new_v = 0.0
            for sp in range(env.n_states):
                prob = env.p(sp, s, a)
                rew = env.r(sp, s, a)
                new_v += prob * (rew + gamma * V[sp])
            V[s] = new_v
            delta = max(delta, abs(v - new_v))
        if delta < theta:
            break
    return V
```

4.2 Policy improvement

For each state s compute action-values $Q(s,a)$ and pick argmax (break ties randomly).

python

 Copy code

```
def policy_improvement(env, V, gamma=0.9):
    policy = np.zeros(env.n_states, dtype=int)
    for s in range(env.n_states):
        qvals = np.zeros(4)
        for a in range(4):
            for sp in range(env.n_states):
                prob = env.p(sp, s, a)
                rew = env.r(sp, s, a)
                qvals[a] += prob * (rew + gamma * V[sp])
            # random tie break
        best_actions = np.where(qvals == qvals.max())[0]
        policy[s] = np.random.choice(best_actions)
    return policy
```

4.3 Policy iteration

Alternate eval/improve until policy stable.

4.4 Value iteration

Synchronous updates with max over actions.

Implement both and record iteration counts (question in assignment).

5. Tabular model-free algorithms (tabular_rl.py)

5.1 Common points

- Use episodic loop: for episode in range(max_episodes) :
 - reset env, accumulate discounted return (or sum of rewards).
 - epsilon and learning rate should **linearly decay** from initial to 0.
 - Use `epsilon_greedy(Q, state, epsilon)` that breaks ties randomly.

5.2 Sarsa (on-policy)

- Maintain $Q(s,a)$ table shape `(n_states, 4)` initialized zeros.
- For each step: choose action A with epsilon-greedy, take step, observe R and next state S' , choose A' with epsilon-greedy, update:

```
Q[s,a] += eta * (R + gamma * Q[s',a'] - Q[s,a])
```

- If `done` : update using `R` only.

5.3 Q-learning (off-policy)

- Update rule:

powershell

 Copy code

```
Q[s,a] += eta * (R + gamma * max_a' Q[s',a'] - Q[s,a])
```

5.4 Return tracking

- For plotting, compute per-episode *discounted return*: $G = \sum_{t=0..T-1} \gamma^t * r_t$.
- Save `returns[episode] = G`.

6. Linear function approximation (linear_rl.py)

6.1 LinearWrapper

- `encode_state(s)` returns array shape `(4, n_features)` or stacked into `(n_features_total,)` depending on design. Assignment suggests one-hot per `(s,a)` (so feature length = `n_states * 4`).
- Implement theta vector length `n_states * 4`.
- `q(s,a) = dot(theta, feature(s,a))`.

Key: For one-hot features the linear update reduces to tabular update — show equivalence in the report.

6.2 Linear Sarsa / Q

- Implement same update rules but compute `q` via `theta` and features.
- Learning rate must be scaled appropriately (the same as tabular if features are one-hot).

7. Deep Q-Network (dqn.py)

This is the most code-heavy piece. I'll give a clean, minimal DQN implementation suitable for FrozenLake images.

7.1 FrozenLakeImageWrapper

- For each tile produce 4 channels: agent position, start map, holes map, goal map.
- Input shape: (4, H, W) for a single sample. Convert to torch tensor with `float32`.

python

 Copy code

```
import torch
import numpy as np

class FrozenLakeImageWrapper:
    def __init__(self, env, h, w):
        self.env = env
        self.h = h
        self.w = w

    def state_to_image(self, state):
        # produce channels
        ch_agent = np.zeros((self.h, self.w), dtype=np.float32)
        ch_start = np.zeros_like(ch_agent)
        ch_holes = np.zeros_like(ch_agent)
        ch_goal = np.zeros_like(ch_agent)
        if state != self.env.absorbing:
            r, c = state // self.env.ncols, state % self.env.ncols
            ch_agent[r, c] = 1.0
        # fill start/hole/goal channels from env.lake
        for i, tile in enumerate(self.env.lake):
            r, c = i // self.env.ncols, i % self.env.ncols
            if tile == 'S':
                ch_start[r, c] = 1.0
            elif tile == 'H':
                ch_holes[r, c] = 1.0
            elif tile == 'G':
                ch_goal[r, c] = 1.0
        return np.stack([ch_agent, ch_start, ch_holes, ch_goal], axis=0)
```

7.2 ReplayBuffer

- Simple ring buffer storing tuples (state_img, action, reward, next_state_img, done) and sampling uniformly.

7.3 DeepQNetwork (PyTorch)

- Small conv net: Conv2d -> ReLU -> flatten -> Linear -> ReLU -> Linear(output 4).
- `forward(x)` returns Q-values per action.

7.4 train_step(batch)

- Compute current Q for actions via `q_values.gather(1, actions)`.
- Compute next target Q via target network: `next_q = target_net(next_states).max(1)[0]` then `target = rewards + gamma * (1 - done) * next_q`.
- MSE loss between q and target (detach target).
- `optimizer.zero_grad(); loss.backward(); optimizer.step()`.

7.5 Training loop

- For episodes:
 - Reset env, get state image.
 - For each step until done:
 - Select action via epsilon-greedy from `online_net`.
 - Step env -> reward, next_state.
 - Store to replay. If buffer $\geq$ batch_size: sample and `train_step`.
 - Periodically copy `online_net` weights to `target_net` every `target_update_freq` episodes or steps.

Teacher tip: Keep networks small — FrozenLake is tiny. DQN is slower and may not beat tabular algorithms on this discrete small env, but implement to satisfy assignment.

8. Plotting & experiments (utils.py)

8.1 Plot returns

- Use `matplotlib` and plot moving average (window=20). Do not specify special colors (assignment/web.run constraint earlier; here it's fine — but default colors ok).

```
def moving_average(x, w=20):
    if len(x) < w:
        return x
    return np.convolve(x, np.ones(w)/w, mode='valid')
```

Save plots as PNG and include in the report.

8.2 Logging

- Save runs (Q-table, theta, model state_dicts) using `np.save` or `torch.save`.

9. main.py: reproducible experiment runner and output.txt

main.py should:

1. Set seeds for reproducibility (python random, numpy, torch).
2. Instantiate env with slip value(s) required.
3. Run model-based algorithms: print final value function or iteration counts.
4. Run Sarsa, Q-Learning, linear versions, DQN — save returns arrays to disk and plot them.
5. Print the requested outputs exactly as the assignment expects and write the same lines to `output.txt`.

Example at start of `main.py`:

python

 Copy code

```
if __name__ == "__main__":
    import random, numpy as np
    random.seed(0); np.random.seed(0)
    # create env
    env = FrozenLake(lake, 4, 4, slip=0.1, seed=0)
    # policy iteration
    pi, V = policy_iteration(env, gamma=0.9)
    # print and write to output.txt
    with open('output.txt', 'w') as f:
        print("Policy iteration complete", file=f)
```

```
print("Policy:", pi, file=f)
# run sarsa, qlearning etc.
```

Teacher tip: Make the `main` produce the exact output the grader expects — check assignment wording for formatting.

10. Report structure and what to write (≤ 5 pages)

Write a concise, clear report. Use this outline:

1. **Title & authors** (1 line)
 2. **Implementation summary** (0.5 page):
 - File list and brief responsibilities.
 - Any deviations (e.g., indexing choice).
 3. **Part 1: Results and experiments** (2–2.5 pages):
 - Q1: iteration counts for policy & value iteration (table).
 - Q2: Plot moving average returns for Sarsa, Q, linear Sarsa, linear Q, DQN (1 figure with subplots or multiple small figures).
 - Q3: hyperparameter tuning — grid used & best pair; brief table of results.
 - Q4–Q6: short theory answers (one paragraph each).
 4. **Part 2: Extra experiment** (1 page):
 - Environment chosen, algorithm used, results, and interpretation.
 5. **Reproducibility:**
 - How to run `main.py` exactly, dependencies and seeds.
 6. **Appendix (if needed):** specific parameter settings.
-

11. Important hyperparameters & recommended defaults

- `gamma = 0.9`
- Tabular Sarsa & Q:
 - `max_episodes = 4000`
 - `eta_initial = 0.5` (linearly to 0)
 - `epsilon_initial = 0.5` (linearly to 0)
- Linear versions: same `eta`, features normalized if needed.
- DQN:

- `learning_rate = 1e-3`
 - `epsilon = 0.2` (decay optional)
 - `batch_size = 32`
 - `buffer_size = 1000`
 - `target_update_frequency = 4` (episodes or steps)
 - small conv channels (4), fc size small (8)
-

12. Debugging checklist (teacher cues)

- If model-based outputs wrong: compare `env.p` with `p.npy`.
 - If Q-values not learning: check epsilon-greedy tie-breaking and action selection, ensure learning rate decays correctly.
 - If linear and tabular disagree: verify one-hot mapping indices match (feature index = $s*4 + a$).
 - If DQN unstable: reduce learning rate, ensure `done` masking when computing `next_q : next_q * (1 - done)`.
 - If plots look noisy: plot moving average window 20 and include raw returns in appendix.
-

13. Quick minimal submission to get marks (if pressed for time)

If you must turn something in quickly, ensure:

1. FrozenLake `env` correct & validated with `p.npy`. (Essential)
2. Implement policy iteration and value iteration; produce iteration counts. (Model-based)
3. Implement tabular Sarsa & Q-learning and save `returns` arrays and moving average plots.
4. Main writes `output.txt` containing those key printouts.
5. Short report answering Q1-Q3 and including the plots.

This will give you a solid, graded submission even if linear and DQN remain incomplete.

14. Example small test & run workflow (what to do now — day-by-day plan)

Day 1 (today)

- Create project structure.
- Implement `FrozenLake` (`p`, `r`).
- Validate `p` against `p.npy`.
- Implement simple `render()` and `step()`.

Day 2

- Implement policy evaluation, improvement, policy iteration, value iteration.
- Run on small lake and big lake, record iteration counts.

Day 3

- Implement tabular Sarsa and Q-Learning (with decays).
- Run on small lake, produce return curves.

Day 4

- Implement `LinearWrapper` and linear algorithms; verify equivalence to tabular on one-hot features.

Day 5-6

- Implement DQN (image wrapper, networks, replay). Run trials (longer training time). Take plots/screenshots.

Day 7

- Final experiments, tune a couple of hyperparameters, produce final plots.

Day 8

- Write the report ≤ 5 pages; prepare `output.txt`; final checks.

15. Handy code snippets (copy-paste)

epsilon-greedy with random tie breaking

python

 Copy code

```
def epsilon_greedy(Q, state, epsilon):
```